Universidad Nacional Autónoma de México
Facultad de Ciencias



Inteligencia Artificial

Laboratorio 2.3

Docente: *Dra. JessicaSarahiMéndezRincón*

Alumna: *OsorioLopezClaudiaYarinet*

Fecha de realización: 21/09/2026

Este código utiliza el algoritmo de poda alfa-beta en el juego del tic-tac-toe. Puedes probarlo tal como está.

https://www.youtube.com/watch?v=I0y-TGehf-4

```
import math

def imprimir_tablero(tablero):
    for fila in tablero:
        print(" ".join(fila))
    print()

def es_tablero_lleno(tablero):
    for fila in tablero:
        if " " in fila:
            return False
    return True

def evaluar_estado(tablero):
    # Evaluación simple para el tic-tac-toe
    ganador = None
    # Filas
    for fila in tablero:
        if fila.count("X") == 3:
            ganador = "X"
        elif fila.count("O") == 3:
            ganador = "O"
    # Columnas
    for col in range(3):
        if tablero[0][col] == tablero[1][col] == tablero[2][col] and tablero[0][col] != " ":
            ganador = tablero[0][col]
    # Diagonales
    if tablero[0][0] == tablero[1][1] == tablero[2][2] and tablero[0][0] != " ":
        ganador = tablero[0][0]
    if tablero[0][2] == tablero[1][1] == tablero[2][0] and tablero[0][2] != " ":
        ganador = tablero[0][2]

    if ganador == "X":
        return 1
    elif ganador == "O":
        return -1
    elif es_tablero_lleno(tablero):
        return 0
    else:
        return None
```

```
def alphabeta(tablero, profundidad, alfa, beta, es_maximizando):
    estado_actual = evaluar_estado(tablero)

    if estado_actual is not None:
        return estado_actual

    if es_maximizando:
        mejor_valor = -math.inf
        for i in range(3):
            for j in range(3):
                if tablero[i][j] == " ":
                    tablero[i][j] = "X"
                    valor = alphabeta(tablero, profundidad + 1, alfa, beta, False)
                    tablero[i][j] = " "
                    mejor_valor = max(mejor_valor, valor)
                    alfa = max(alfa, mejor_valor)
                    if beta <= alfa:
                        break
        return mejor_valor
    else:
        mejor_valor = math.inf
        for i in range(3):
            for j in range(3):
                if tablero[i][j] == " ":
                    tablero[i][j] = "O"
                    valor = alphabeta(tablero, profundidad + 1, alfa, beta, True)
                    tablero[i][j] = " "
                    mejor_valor = min(mejor_valor, valor)
                    beta = min(beta, mejor_valor)
                    if beta <= alfa:
                        break
        return mejor_valor

def mejor_movimiento(tablero):
    mejor_valor = -math.inf
    mejor_mov = None
    alfa = -math.inf
    beta = math.inf

    for i in range(3):
        for j in range(3):
            if tablero[i][j] == " ":
                tablero[i][j] = "X"
                valor = alphabeta(tablero, 0, alfa, beta, False)
                tablero[i][j] = " "

                if valor > mejor_valor:
                    mejor_valor = valor
                    mejor_mov = (i, j)
                alfa = max(alfa, mejor_valor)

    return mejor_mov

# Ejemplo de uso con tic-tac-toe
tablero_inicial = [
    [" ", " ", " "],
    [" ", " ", " "],
    [" ", " ", " "]
]

turno = "X"

while evaluar_estado(tablero_inicial) is None and not es_tablero_lleno(tablero_inicial):
    imprimir_tablero(tablero_inicial)

    if turno == "X":
        fila, col = map(int, input("Turno de X. Ingrese fila y columna (separadas por espacio): ").split())
        if tablero_inicial[fila][col] == " ":
            tablero_inicial[fila][col] = "X"
        else:
            print("Casilla ocupada. Intenta de nuevo.")
            continue
    else:
        print("Turno de O (IA)")
        movimiento = mejor_movimiento(tablero_inicial)
        fila, col = movimiento
```

```
        tablero_inicial[fila][col] = "O"

    turno = "O" if turno == "X" else "X"

imprimir_tablero(tablero_inicial)
resultado = evaluar_estado(tablero_inicial)

if resultado == 1:
    print("¡X gana!")
elif resultado == -1:
    print("¡O gana!")
else:
    print("¡Empate!")
```

```
Turno de X. Ingrese fila y columna (separadas por espacio): 0 0
X



Turno de O (IA)
X O



Turno de X. Ingrese fila y columna (separadas por espacio): 0 2
X O X



Turno de O (IA)
X O X
O


Turno de X. Ingrese fila y columna (separadas por espacio): 2 2
X O X
O
    X

Turno de O (IA)
X O X
O O
    X

Turno de X. Ingrese fila y columna (separadas por espacio): 1 2
X O X
O O X
    X

¡X gana!
```

Ejercicio adicional:

Modifica el código para que sea aplicable a un juego diferente o expande el código para permitir que dos jugadores humanos jueguen entre sí. También puedes experimentar con diferentes heurísticas o ajustar los valores de alfa y beta para ver cómo afectan el rendimiento del algoritmo.

```
import math
import time

# FUNCION: imprimir_tablero
# Muestra el estado actual del tablero en consola

def imprimir_tablero(tablero):
    for fila in tablero:
        print(" ".join(fila))   # imprime cada fila separando los elementos con espacios
    print()


# ----------------------------------------------------------
# FUNCION: es_tablero_lleno
# Verifica si ya no hay espacios disponibles en el tablero
# Retorna:
```

```
# True  -> si el tablero esta lleno
# False -> si aún quedan movimientos posibles
# ------------------------------------------------------------
def es_tablero_lleno(tablero):
    for fila in tablero:
        if " " in fila:
            return False
    return True


# ------------------------------------------------------------
# FUNCION: evaluar_estado
# Determina si existe un ganador o si el juego terminó en empate.
# Se utiliza como función de evaluación del algoritmo minimax.
#
# Parámetros:
# tablero      -> estado actual del tablero
# profundidad  -> nivel actual del árbol de búsqueda
#
# Retorna:
# valor positivo  -> gana X
# valor negativo  -> gana O
# 0               -> empate
# None            -> el juego continúa
# ------------------------------------------------------------
def evaluar_estado(tablero, profundidad):

    ganador = None


    # Evaluar filas

    for fila in tablero:
        if fila.count("X") == 3:
            ganador = "X"
        elif fila.count("O") == 3:
            ganador = "O"


    # Evaluar columnas

    for col in range(3):
        if tablero[0][col] == tablero[1][col] == tablero[2][col] and tablero[0][col] != " ":
            ganador = tablero[0][col]

    # Evaluar diagonales

    if tablero[0][0] == tablero[1][1] == tablero[2][2] and tablero[0][0] != " ":
        ganador = tablero[0][0]

    if tablero[0][2] == tablero[1][1] == tablero[2][0] and tablero[0][2] != " ":
        ganador = tablero[0][2]


    # Asignar valores heurísticos
    # Se considera la profundidad para favorecer
    # victorias más rápidas o derrotas más tardías

    if ganador == "X":
        return 10 - profundidad

    elif ganador == "O":
        return profundidad - 10

    elif es_tablero_lleno(tablero):
        return 0

    else:
        return None


# ------------------------------------------------------------
# FUNCION: alphabeta
# Implementación del algoritmo Minimax con poda Alpha-Beta.
#
# Parámetros:
# tablero        -> estado actual del tablero
# profundidad    -> nivel del árbol de búsqueda
```

```
# alfa           -> mejor valor encontrado para MAX
# beta           -> mejor valor encontrado para MIN
# es_maximizando -> indica si el turno es del jugador MAX (X)
#
# Retorna:
# valor heurístico del estado evaluado
# ------------------------------------------------------
def alphabeta(tablero, profundidad, alfa, beta, es_maximizando):

    estado_actual = evaluar_estado(tablero, profundidad)

    # Caso base: el juego terminó
    if estado_actual is not None:
        return estado_actual


    # Jugador MAX (X) intenta maximizar el valor

    if es_maximizando:

        mejor_valor = -math.inf

        for i in range(3):
            for j in range(3):

                if tablero[i][j] == " ":

                    tablero[i][j] = "X"

                    valor = alphabeta(tablero, profundidad + 1, alfa, beta, False)

                    tablero[i][j] = " "

                    mejor_valor = max(mejor_valor, valor)

                    alfa = max(alfa, mejor_valor)

                    # Condición de poda Alpha-Beta
                    if beta <= alfa:
                        break

        return mejor_valor

    # Jugador MIN (O) intenta minimizar el valor

    else:

        mejor_valor = math.inf

        for i in range(3):
            for j in range(3):

                if tablero[i][j] == " ":

                    tablero[i][j] = "O"

                    valor = alphabeta(tablero, profundidad + 1, alfa, beta, True)

                    tablero[i][j] = " "

                    mejor_valor = min(mejor_valor, valor)

                    beta = min(beta, mejor_valor)

                    # Condición de poda Alpha-Beta
                    if beta <= alfa:
                        break

        return mejor_valor


# ------------------------------------------------------
# FUNCION: mejor_movimiento
# Calcula la mejor jugada posible para el jugador X
# utilizando el algoritmo Alpha-Beta.
#
# También mide el tiempo de ejecución del algoritmo
```

```
# para analizar su rendimiento.
# ------------------------------------------------------------
def mejor_movimiento(tablero):

    inicio = time.time()

    mejor_valor = -math.inf
    mejor_mov = None

    alfa = -math.inf
    beta = math.inf

    for i in range(3):
        for j in range(3):

            if tablero[i][j] == " ":

                tablero[i][j] = "X"

                valor = alphabeta(tablero, 0, alfa, beta, False)

                tablero[i][j] = " "

                if valor > mejor_valor:
                    mejor_valor = valor
                    mejor_mov = (i, j)

                alfa = max(alfa, mejor_valor)

    fin = time.time()

    print("Tiempo de decisión de la IA:", fin - inicio, "segundos")

    return mejor_mov


# PROGRAMA PRINCIPAL
# Inicialización del tablero de Tic-Tac-Toe

tablero = [
    [" ", " ", " "],
    [" ", " ", " "],
    [" ", " ", " "]
]

# Selección del modo de juego
print("Seleccione modo de juego")
print("1 - Humano vs IA")
print("2 - Humano vs Humano")

modo = int(input("Modo: "))

turno = "X"


# BUCLE PRINCIPAL DEL JUEGO
# Se ejecuta hasta que exista un ganador o empate

while evaluar_estado(tablero, 0) is None and not es_tablero_lleno(tablero):

    imprimir_tablero(tablero)

    if turno == "X":

        fila, col = map(int, input("Turno de X (fila columna): ").split())

        if tablero[fila][col] == " ":
            tablero[fila][col] = "X"
        else:
            print("Casilla ocupada")
            continue

    else:

        if modo == 1:

            print("Turno de O (IA)")
            fila, col = mejor_movimiento(tablero)
```

```
            fila, col = mejor_movimiento(tablero)
            tablero[fila][col] = "O"

        else:

            fila, col = map(int, input("Turno de O (fila columna): ").split())

            if tablero[fila][col] == " ":
                tablero[fila][col] = "O"
            else:
                print("Casilla ocupada")
                continue

    # Cambio de turno
    turno = "O" if turno == "X" else "X"

# RESULTADO FINAL DEL JUEGO
imprimir_tablero(tablero)

resultado = evaluar_estado(tablero, 0)

if resultado > 0:
    print("X gana")
elif resultado < 0:
    print("O gana")
else:
    print("Empate")
```

```
Seleccione modo de juego
1 - Humano vs IA
2 - Humano vs Humano
Modo: 1



Turno de X (fila columna): 0 0
X



Turno de O (IA)
Tiempo de decisión de la IA: 0.01478266716003418 segundos
X O



Turno de X (fila columna): 0 2
X O X



Turno de O (IA)
Tiempo de decisión de la IA: 0.0007321834564208984 segundos
X O X
  O


Turno de X (fila columna): 2 1
X O X
  O
  X

Turno de O (IA)
Tiempo de decisión de la IA: 0.0001308917999267578 segundos
X O X
  O
O X

Turno de X (fila columna): 1 2
X O X
  O X
O X

Turno de O (IA)
Tiempo de decisión de la IA: 4.3392181396484375e-05 segundos
X O X
  O X
O X O

Turno de X (fila columna): 1 0
X O X
```

```
X O X
O X O

Empate
```

Conclusiones finales del estudiante

Haz doble clic (o ingresa) para editar

© 2026 UNAM Facultado de Ciencias – Todos los derechos reservados